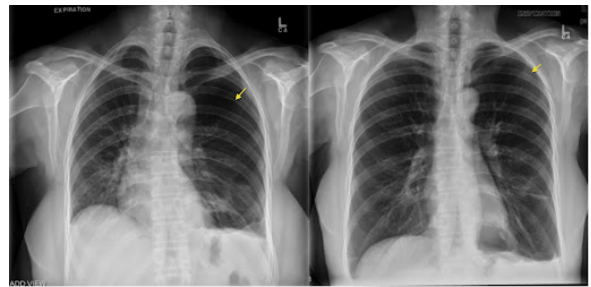
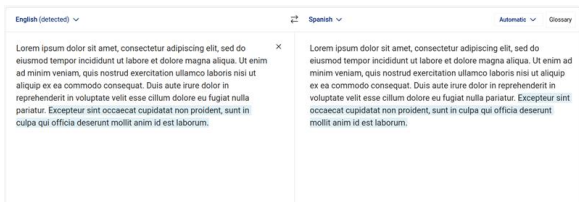
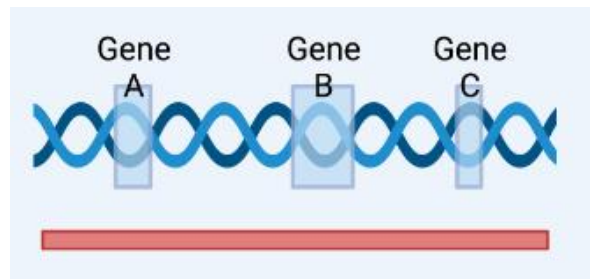


AICE3002 Introduction to Deep Learning

AICE6001 Introduction to Deep Learning (MSc)

Jonathon Hare and Antonia Marcu

What does Deep Learning (and Differentiable Programming) give us?



Deep Learning and Differentiable Programming give us the ability to computationally solve problems without needing to handcraft the solution or engineer the features. As long as we have data we can build and train models to make predictions.

Machine Learning in a Nutshell

Data	$\{\mathbf{x}_n, \mathbf{y}_n\}_{n=1}^N$ $\{\mathbf{x}_n\}_{n=1}^N$
Function Approximator	$\mathbf{y} = f(\mathbf{x}; \boldsymbol{\theta})$
Loss function	$\mathcal{L} = \sum_{n=1}^N \ \mathbf{y}_n - f(\mathbf{x}_n; \boldsymbol{\theta})\ _2^2$
Parameter Estimation	$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \mathcal{L}$
Prediction	$\hat{\mathbf{y}}_{N+1} = f(\mathbf{x}_{N+1}, \hat{\boldsymbol{\theta}})$

What is Deep Learning?

Deep learning is primarily characterised by function compositions:

- Feedforward networks: $\mathbf{y} = f(g(\mathbf{x}; \boldsymbol{\theta}_g); \boldsymbol{\theta}_f)$
 - Often with relatively simple functions (e.g. $f(\mathbf{x}; \boldsymbol{\theta}_f) = \sigma(\mathbf{x}^\top \boldsymbol{\theta}_f)$)

- Recurrent networks: $\mathbf{y}_t = f(\mathbf{y}_{t-1}, \mathbf{x}_t; \boldsymbol{\theta}) = f(f(\mathbf{y}_{t-2}, \mathbf{x}_{t-1}; \boldsymbol{\theta}), \mathbf{x}_t; \boldsymbol{\theta}) = \dots$

In the early days the focus of deep learning was on learning functions for classification. Nowadays the functions are much more general in their inputs and outputs.

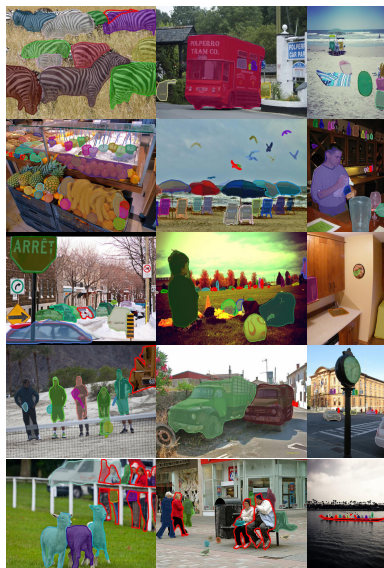
What is Differentiable Programming?

- Differentiable programming is a term coined by Yann Lecun to describe a superset of Deep Learning.
- Captures the idea that computer programs can be constructed of parameterised functional blocks in which the parameters are learned using some form of gradient-based optimisation.
- So we need to be able to compute gradients with respect to the parameters of these functional blocks.
- The functional blocks don't need to be direct functions in a mathematical sense; more generally they can be *algorithms*.
- What if the functional block we're learning parameters for is itself an algorithm that optimises the parameters of an internal algorithm using a gradient based optimiser?!¹

Is all Deep Learning Differentiable Programming?

- Not necessarily!
 - Most deep learning systems are trained using first order gradient-based optimisers, but there is an active body of research on gradient-free methods.
 - There is an increasing interest in methods that use different styles of learning, such as Hebbian learning, within deep networks. More broadly there are a number of us² who are interested in biologically motivated models and learning methods.
 - * There's a lot of recent research that computes biological proxies for gradients though!
 - This course will primarily focus on differentiable methods, but we'll look at how relaxations can be made to make non-differentiable operators learnable with gradient-based optimisers.

Success stories - Object detection and segmentation

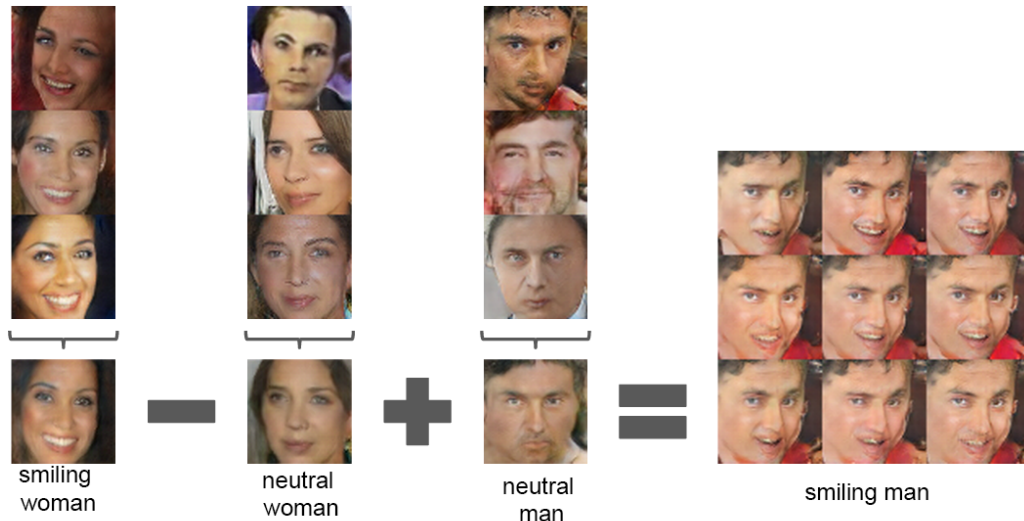


Pinheiro, Pedro O., et al. "Learning to refine object segments." European Conference on Computer Vision. Springer, 2016.

¹See our ICLR 2019 paper: <https://arxiv.org/abs/1812.03928> and NeurIPS 2019 paper: <https://arxiv.org/abs/1906.06565>

²including at least myself, my PhD students and Geoff Hinton!

Success stories - Image generation



Radford, Alec, Luke Metz, and Soumith Chintala. "Unsupervised representation learning with deep convolutional generative adversarial networks." arXiv preprint arXiv:1511.06434 (2015).

Success stories - Translation

GPT-4

Prompt:

Translate the following Irish text to English:

"Bhí mé ag siúl sa choill nuair a chuala mé ceol álainn ón adharc."

Output (With LLM Integration):

I was walking in the forest when I heard beautiful music from the horn.

Output (Traditional MT):

I was in the wood when I heard nice sound from the horn.

Lyu, C., Du, Z., Xu, J., Duan, Y., Wu, M., Lynn, T., Aji, A.F., Wong, D.F., Liu, S. and Wang, L., 2023. A Paradigm Shift: The Future of Machine Translation Lies with Large Language Models. LREC-COLING 2024

Language translation has made massive leaps over the last 10 years as a result of deep learning. Initially this was because of large aligned datasets and translation models that targeted transforming a sequence of words in one language to another. As a result of large language models we are increasingly seeing translation quality improve in terms of the nuance and subjective feel of the translation — translations are starting to becoming more natural and less mechanistic (but possibly less objectively accurate as a result).

Why should we care about this?

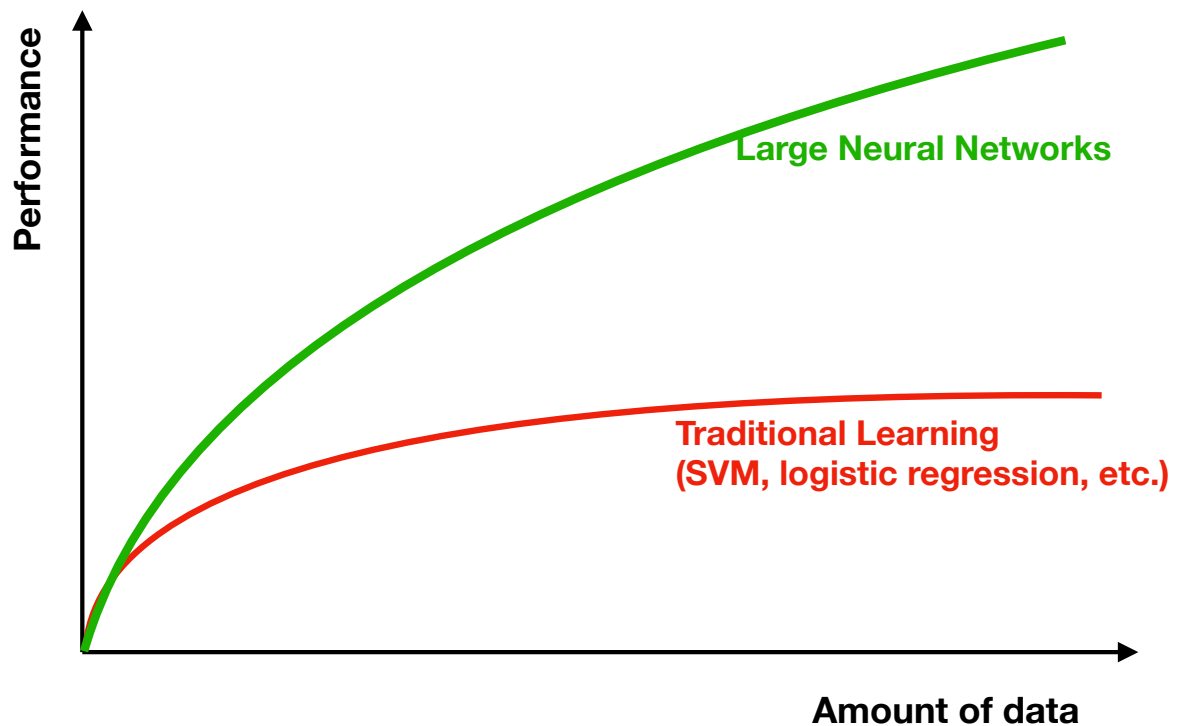


Image courtesy of Andrew Ng

The above picture shows how model performance has been shown to be linked to the amount of data. The story of the last 10-15 years has been one of increasingly growing data. Traditional approaches to machine learning have largely not scaled in performance in the same way large deep neural networks have. This is predominantly because of the way features are extracted from the data; traditional machine learning often used hand-crafted features, whereas the story of deep learning is all about feature learning — giving the model essentially raw data and letting it learn to appropriate features to make decisions from.

We should recognise however that traditional methods still have their place - when datasets are small, and perhaps there isn't much feature engineering to do, they can and do outperform deeper models. At the same time, a large proportion of deep network architectures are themselves nothing more than a learned linear model (linear regression, logistic regression, SVM, ...) attached to a learned feature extractor, so those very basic traditional methods are still very much a big part of the picture today!

Key takeaways

- **More data availability** in all domains...
- ...allows for **bigger/deeper/more complex models**
- ...models that can **learn the features**
- and these can have far **greater performance**

What is the objective of this module?

A word of warning: This is not a module about how to apply someone else's deep network architecture to a task, or how to train existing models! [1em] You will learn some of that along the way of course, but the real objective is for you to graduate understanding how deep learning works beneath the hood.

What is the objective of this module?

- Understand the underlying mathematical and algorithmic principles of deep learning.
- Apply deep learning methods to real datasets.
- Learn how to train models using deep learning libraries.
- Think critically about why certain approaches were designed in the way they were.

How is this module going to be delivered?

- Lectures and seminars (3 most weeks)
 - This is a new module; we're utilising some material from our older module, but also adding/expanding and changing. Expect the moodle pages to change!
 - You need to read the suggested papers/links before the lectures!
 - There is maybe a little room for some flexibility later in the course on topics - tell us what you're interested in!
 - Lectures will be face to face, but also recorded for the website when possible (do not rely on this!).

How is this module going to be delivered?

- Seminars
 - We'll look at a particular paper, papers or ideas in some detail and have an open discussion
 - You'll need to prepare in advance & be ready to ask questions and share your thoughts
 - Not recorded, but contents examinable

How is this module going to be delivered?

- Labs (1x 2 hour session per week for 8 weeks + one additional optional help / catchup session)
 - Labs consist of a number of Jupyter notebooks you will work through.
 - * You'll be using PyTorch as the primary framework, with Torchbearer to help out.
 - * You will need to utilise GPU-compute for the later labs (we provide Google Colab links so you can use NVidia GPUs in the cloud).
 - Labs are in-person (AICE lab in B32) with a team of PhD student demonstrators & both of us.
 - Please ask lots of questions and use this time to get help on the labs.
 - After each block of labs you will have to do a follow-up "Lab Investigation" that will be marked.

WEEK PLANNER						2026-27
Modules: AICE3002 / AICE6001						
Week	Monday	Tuesday	Wednesday	Thursday	Friday	
1	10:00-11:00 (L/R F) Introduction			12:00-13:00 (L/R 1) Vector/matrix calculus, functions, ODEs, and ODEs		
2	10:00-11:00 (L/R F) Machine computation, the Hough Challenge, and data asymmetry	Lab 1: Tensor Computing gradients 1	12:00-13:00 (L/R 1) Computing gradients 1			
3	10:00-11:00 (L/R F) Backpropagation	Lab 2: Automatic Differentiation	12:00-13:00 (L/R 1) Gradient Descent			
4	10:00-11:00 (L/R F) Fundamentals review	Lab 3: Optimisation	12:00-13:00 (L/R 1) Basic ML			
5	10:00-11:00 (L/R F) Going deep	Lab 4: Linear Models	12:00-13:00 (L/R 1) Practical training	Lab 102 submission		
6	10:00-11:00 (L/R F) Convolution fundamentals	Lab 5: MLP	12:00-13:00 (L/R 1) CNNs I			
7	10:00-11:00 (L/R F) CNNs II	Lab 6: CNN	12:00-13:00 (L/R 1) CNNs II			
8	10:00-11:00 (L/R F) Transfer Learning	Lab 7: Transfer Learning	12:00-13:00 (L/R 1) Transfer Learning			
9	10:00-11:00 (L/R F) XXX	Consolidation Lab Dropin	12:00-13:00 (L/R 1) XXX			
10	10:00-11:00 (L/R F) LSTM	Lab 8: Sequence Models	12:00-13:00 (L/R 1) Sequence Models			
11	10:00-11:00 (L/R F) Autoencoders		12:00-13:00 (L/R 1) XXX			
12	10:00-11:00 (L/R F) Reinforce		12:00-13:00 (L/R 1) XXX			
13	CHRISTMAS					
14	SEMESTER 1 EXAMS					

What will we cover in the module?

AICE3002: <https://moodle.ecs.soton.ac.uk/course/view.php?id=1031> [1em] AICE6001: <https://moodle.ecs.soton.ac.uk/course/view.php?id=1034>

Lab session plan

Lab	Date	Topic
Lab 1	30-Sep	Tensor programming with PyTorch
Lab 2	07-Oct	Automatic Differentiation
Lab 3	14-Oct	Optimisation
Lab 4	21-Oct	Linear Models
Lab 5	28-Oct	NNs with PyTorch and Torchbearer
	04-Nov	(Catch up / help)
Lab 6	11-Nov	CNNs with PyTorch and Torchbearer
Lab 7	18-Nov	Using pretrained models and transfer learning
Lab 8	25-Nov	Recurrent Networks, Sequence Prediction & Embeddings

Assessment Structure

- Lab Investigations 40%
- Final exam 60%

Lab Investigations

Note these are the same for both AICE3002 and AICE6001

Investigation	Submission Date	Labs covered
1 - Gradients and Matrix Completion	23rd Oct, 4PM	1 and 2
2 - Optimisation and Wide Networks	20th Nov, 4PM	3, 4 and 5
3 - Convolutions and Transfer	27th Nov, 4PM	6 and 7
4 -Transforming Sequences	11th Dec, 4PM	8

Exam

- 2 hour duration, in-person, written.
- Common Section A questions.
- Different Section B questions for AICE3002 and AICE6001.
- More details to follow later in the semester, including a cut-down sample paper with representative questions

What do we expect you already know?

- You have have taken Foundations of Machine Learning, or are currently taking it.
- Fundamentals of:
 - Matrix Algebra (matrix-matrix, matrix-vector and matrix-scalar operations, inverse, determinant, rank, Eigendecomposition, SVD);
 - Probability & Statistics (1st-order summary statistics, simple continuous and discrete probability distributions, expected values, etc); and,
 - Multivariable Calculus (partial differentiation, chain-rule).
- Programming in Python

What might you already know?

- How to use a deep learning framework (Keras, Tensorflow, PyTorch)?
- How to train an existing model architecture using a GPU?
- How to perform transfer learning?
- How to perform differentiable sampling of a Multivariate Normal Distribution?

What comes after this module?

- AICE6002: Deep Learning Research
 - taught by both of us
 - focus on translating the fundamentals you learn in this module to cutting edge research
 - plus lectures on advanced architectures, generative models, etc

Homework & questions

- Homework: Register for access to SWARM - <https://tinyurl.com/ecs-swarm>
- Any questions?